PROGRAMMING IN JAVA

ASSIGNMENT 4 – CO4

EVENT-DRIVEN GUI DESIGN

Student Details	**To be completed**
Name	J.SUBHAASHREE
Register Number	192424008
Course / Section	AI&DS
Faculty	Ravi Pujitha
Submission Date	5.11.26
GitHub Repository Link	https://github.com/Subhaashree06/JAVA-Assignments/blob/main/CO4_Java_Assignment4_Code.ipynb

ASSIGNMENT SCOPE AND SUBMISSION REQUIREMENTS

This report follows the uploaded Assignment 4 – CO4 question paper. The assignment is an individual, one-week, 50-mark assessment covering applets, the event delegation model, listeners and adapter classes, inner and anonymous classes, AWT controls, layout managers, menus and dialogs, Swing components, and GUI–JDBC integration.

Task	Topic	Marks	Deliverables in this report
1	GUI design review – fee payment	6	Defect diagnosis, repaired layered design, event handlers, defence and complexity
2	Hospital appointment desk	8	Event table, containment hierarchy, SIMATS-style pseudocode, event trace and caching analysis
3	Campus library search screen	9	Layout selection, nested panel diagram, Swing implementation, resize test plan and GUI mockup
4	Event-handling forensics – drawing pad	7	Defect explanation, failing interaction, completed TODOs, code and BufferedImage comparison
5	Currency converter – four event styles	7	Frame listener, named inner class, anonymous listener, lambda, dialogs, Enter key and comparison
6	Campus event registration application	13	Requirements, screen design, layered class architecture, pseudocode, Swing/JDBC code, SQL and pagination analysis

Evidence note: screenshots of live AWT/Swing windows and actual MySQL execution must be captured on a desktop with a graphical environment and a configured MySQL server. This report includes designed colour mockups and executable source code, but does not claim that a GUI or database was run in this document-generation environment.

REQUIRED EXECUTION ENVIRONMENT

- JDK 17 or later; compile each public class from its own .java file.
- For Tasks 1 and 6, configure MySQL and add MySQL Connector/J to the project classpath.
- Swing GUI programs should be run on a desktop with a display; Google Colab/headless sessions generally cannot show interactive desktop windows.
- Capture screenshots for normal, invalid and duplicate cases and include them in the matching GitHub task folders.

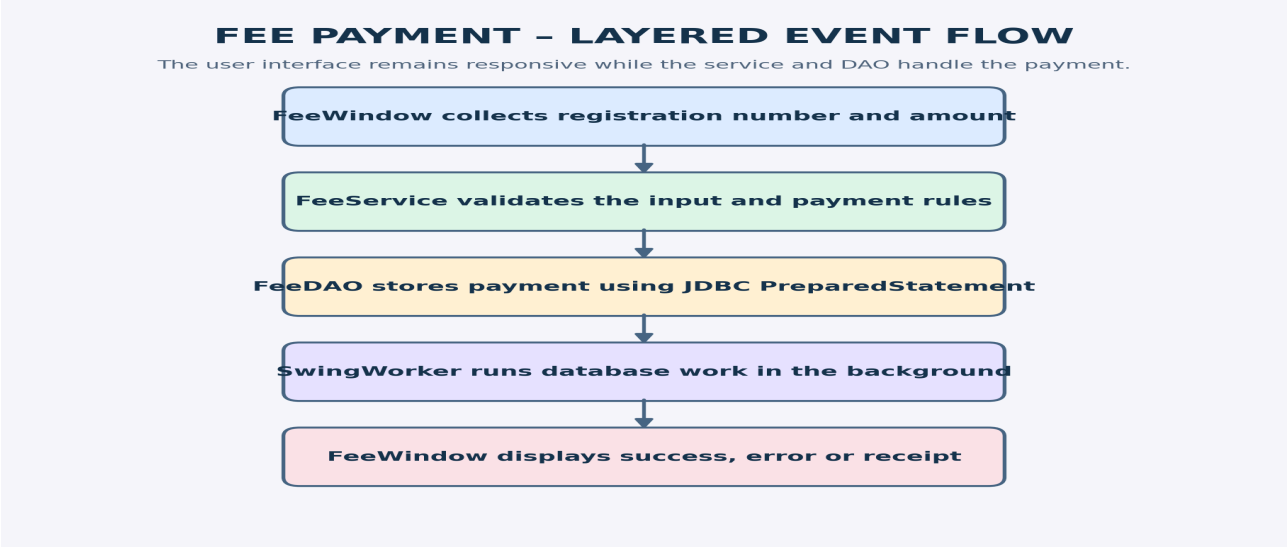
• Use PreparedStatement, try-with-resources, and database constraints; never concatenate untrusted user input into SQL.

TASK 1 – GUI DESIGN REVIEW: FEE PAYMENT WINDOW

Task 1.1 – Diagnose the design

Deficiency	What can go wrong / user experience
UI and business logic are mixed in FeeWindow	The frame performs parsing, database access and payment rules; it is hard to unit-test without opening a window.
String comparison uses == for action command	== compares object references, not string content; the Pay branch may not run reliably. Use equals() or a dedicated Action.
No empty or numeric validation	Empty, alphabetic, negative or malformed amounts can cause NumberFormatException or invalid payment attempts.
Database call runs on the Event Dispatch Thread	The UI freezes while the database responds; repainting and user interaction are blocked.
Thread.sleep(3000) blocks the event thread	The window appears hung for three seconds and cannot respond to clicks or repaint requests.
No visible success/error feedback	System.out.println("Paid") is not useful feedback to the person using the window; receipt details are absent.
Receipt button is not implemented	Clicking Receipt falls into the else branch and clears fields instead of displaying/printing a receipt.
SQL uses string concatenation	A malicious or malformed registration number can alter the SQL; quotes and special characters can break the statement.
Connection and Statement are not closed	Database resources can leak; the program may exhaust connections or statements.
Checked exceptions are not handled	SQLException and InterruptedException are not properly caught or declared; the program may not compile as shown.
No window-closing handling	The user cannot reliably close the frame and pending work is not considered.

Task 1.2 – Repaired class structure



Class	Responsibility
FeeWindow (JFrame)	Collects registration number and amount; displays status and receipt; delegates actions.
FeeService	Validates input and applies fee-payment rules; contains no Swing component code.
FeeDAO	Uses JDBC PreparedStatement and try-with-resources to store the payment.

PaymentResult	Carries transaction ID, amount and payment status back to the UI.

Repaired event handling – FeeWindow.java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.math.BigDecimal;

public class FeeWindow extends JFrame {
    private final JTextField regNo = new JTextField(14);
    private final JTextField amount = new JTextField(10);
    private final JLabel status = new JLabel("Enter details to pay.");
    private final FeeService service = new FeeService();

    public FeeWindow() {
        super("Student Fee Payment");
        setLayout(new BorderLayout(8, 8));
        JPanel form = new JPanel(new GridLayout(2, 2, 6, 6));
        form.add(new JLabel("Registration No:")); form.add(regNo);
        form.add(new JLabel("Amount (INR):")); form.add(amount);
        JButton pay = new JButton("Pay");
        JButton clear = new JButton("Clear");
        JButton receipt = new JButton("Receipt");
        JPanel actions = new JPanel(new FlowLayout());
        actions.add(pay); actions.add(clear); actions.add(receipt);
        add(form, BorderLayout.CENTER); add(actions, BorderLayout.SOUTH);
        add(status, BorderLayout.NORTH);

        pay.addActionListener(e -> payFee());
        clear.addActionListener(e -> { regNo.setText(""); amount.setText("");
status.setText("Cleared."); });
        receipt.addActionListener(e -> JOptionPane.showMessageDialog(this,
                service.getLastReceipt(), "Receipt", JOptionPane.INFORMATION_MESSAGE));
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        pack(); setLocationRelativeTo(null);
    }

    private void payFee() {
        String id = regNo.getText().trim();
        final BigDecimal value;
        try {
            value = new BigDecimal(amount.getText().trim());
            if (id.isEmpty() || value.signum() <= 0) throw new IllegalArgumentException();
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Enter a registration number and positive amount.",
                    "Invalid input", JOptionPane.ERROR_MESSAGE);
            return;
        }
        // Service/database work is delegated to a background worker.
        new SwingWorker<String, Void>() {
            protected String doInBackground() throws Exception {
                return service.pay(id, value);
            }
            protected void done() {
                try { status.setText(get()); }
                catch (Exception ex) {
                    status.setText("Payment failed.");
                    JOptionPane.showMessageDialog(FeeWindow.this, ex.getMessage(),
                            "Payment error", JOptionPane.ERROR_MESSAGE);
                }
```

```
            }
        }.execute();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new FeeWindow().setVisible(true));
    }
}
```

FeeService.java and FeeDAO.java

```
import java.math.BigDecimal;
class FeeService {
    private final FeeDAO dao = new FeeDAO();
    private String lastReceipt = "No payment yet.";
    String pay(String regNo, BigDecimal amount) throws Exception {
        if (regNo == null || regNo.trim().isEmpty())
            throw new IllegalArgumentException("Registration number is required.");
        if (amount == null || amount.signum() <= 0)
            throw new IllegalArgumentException("Amount must be positive.");
        long id = dao.insertPayment(regNo.trim(), amount);
        lastReceipt = "Receipt / Transaction: " + id + "\nRegistration: " + regNo
                + "\nAmount: Rs. " + amount;
        return "Payment saved. Transaction ID: " + id;
    }
    String getLastReceipt() { return lastReceipt; }
}

import java.sql.*;
import java.math.BigDecimal;
class FeeDAO {
    private static final String URL = "jdbc:mysql://localhost:3306/campus";
    private static final String USER = "root";
    private static final String PWD = "CHANGE_ME";
    long insertPayment(String regNo, BigDecimal amount) throws SQLException {
        String sql = "INSERT INTO fees(reg_no, amount) VALUES (?, ?)";
        try (Connection c = DriverManager.getConnection(URL, USER, PWD);
             PreparedStatement ps = c.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, regNo); ps.setBigDecimal(2, amount);
            ps.executeUpdate();
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) return rs.getLong(1);
            }
        }
        throw new SQLException("Payment was not assigned a transaction ID.");
    }
}
```

Database setup for this example: CREATE TABLE fees (payment_id BIGINT PRIMARY KEY AUTO_INCREMENT, reg_no VARCHAR(30) NOT NULL, amount DECIMAL(10,2) NOT NULL CHECK (amount > 0), paid_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP). The MySQL JDBC driver must be on the runtime classpath. Replace the sample connection settings with local credentials; never commit real passwords to GitHub.

Task 1.3 – Design defence

The redesign separates presentation, business rules and persistence. FeeWindow can be tested as a user interface, FeeService can be unit-tested with a mock DAO, and FeeDAO can be tested against a test database. PreparedStatement prevents SQL injection, validation gives clear errors, and SwingWorker keeps slow database operations off the Event Dispatch Thread. Each button has a focused action, making future receipt, refund or payment-provider changes easier to add without rewriting the whole window.

Alternative concept and complexity analysis

Design	Event dispatch time	Space / trade-off
One listener with if–else over k commands	Worst case O(k) string comparisons per event	O(1) listener object; branching grows as buttons are added
One listener/action per button	O(1) direct dispatch to the registered handler	O(k) listener/action objects; clearer responsibilities and simpler extension

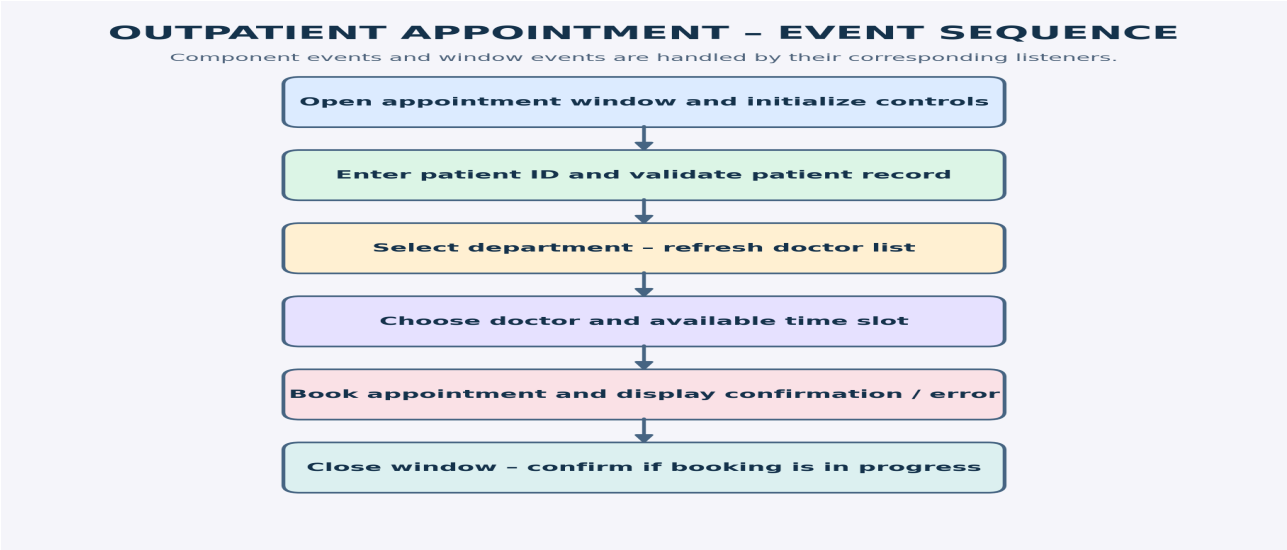
Both approaches use O(1) work for the payment operation itself, excluding database time. The comparison concerns dispatch selection. A Swing Action is preferable when the same command must be reused by a button, menu item and keyboard shortcut.

TASK 2 – FROM REQUIREMENT TO EVENT-DRIVEN DESIGN: HOSPITAL APPOINTMENT DESK

Task 2.1 – Event model

Event source	Event object	Listener interface	Action
Department JComboBox	ItemEvent	ItemListener	Load doctors for selected department and reset doctor selection.
Doctor JComboBox	ActionEvent	ActionListener	Update selected doctor state.
Time-slot JComboBox	ActionEvent	ActionListener	Update selected appointment slot.
Book JButton	ActionEvent	ActionListener	Validate patient, doctor and slot; book appointment.
Clear JButton	ActionEvent	ActionListener	Clear fields and selections.
Window close button	WindowEvent	WindowListener / WindowAdapter	If booking is in progress, ask before closing.
Window opened	WindowEvent	WindowListener / WindowAdapter	Initialize lists and focus patient ID.

Task 2.2 – Component containment tree




```
JFrame AppointmentDesk (BorderLayout)
├── JPanel patientPanel (GridLayout)
│   ├── JLabel "Patient ID"
│   └── JTextField patientId
├── JPanel appointmentPanel (GridBagLayout)
│   ├── JLabel + JComboBox<String> department
│   ├── JLabel + JComboBox<Doctor> doctor
│   └── JLabel + JComboBox<String> timeSlot
└── JPanel actionPanel (FlowLayout)
    ├── JButton Book
    ├── JButton Clear
    └── JLabel status
```

Task 2.3 – SIMATS-style algorithmic pseudocode

Algorithm 1: Refresh doctor list when department changes

```
START
    INPUT selectedDepartment
```

```
    CLEAR doctorComboBox
    IF selectedDepartment IS EMPTY THEN
        DISPLAY "Select a department"
    ELSE
        doctorList ← doctorMap.get(selectedDepartment)
        FOR EACH doctor IN doctorList DO
            ADD doctor TO doctorComboBox
        END FOR
        SET selected doctor TO first available doctor
    END IF
END
```

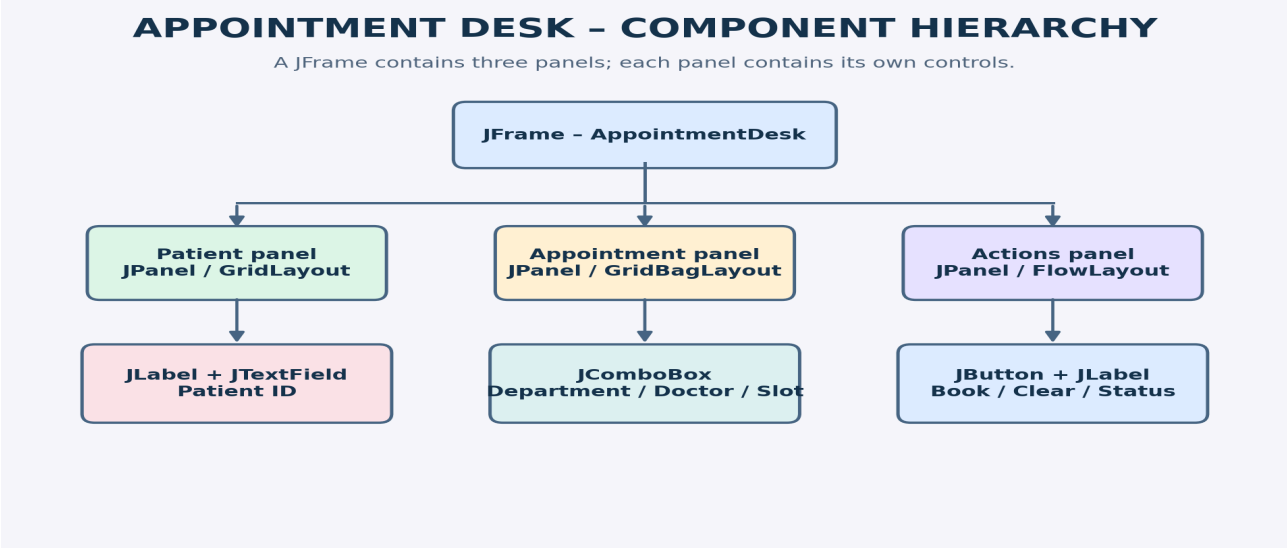
Algorithm 2: Validate and book appointment

```
START
    INPUT patientID, department, doctor, timeSlot
    IF patientID IS EMPTY THEN
        DISPLAY "Patient ID is required"
        STOP
    END IF
    IF patientID DOES NOT EXIST IN patient records THEN
        DISPLAY "Patient not found"
        STOP
    END IF
    IF department OR doctor OR timeSlot IS NOT SELECTED THEN
        DISPLAY "Select department, doctor and time slot"
        STOP
    END IF
    IF timeSlot IS NOT AVAILABLE THEN
        DISPLAY "Choose another time slot"
        STOP
    END IF
    SET bookingInProgress ← TRUE
    SAVE appointment through AppointmentService
    SET bookingInProgress ← FALSE
    DISPLAY confirmation number
END
```

Illustrative Java event-handler fragment

```
departmentCombo.addItemListener(e -> {
    if (e.getStateChange() == ItemEvent.SELECTED) {
        String department = (String) e.getItem();
        doctorCombo.removeAllItems();
        for (Doctor doctor : doctorMap.getOrDefault(department, List.of()))
            doctorCombo.addItem(doctor);
    }
});
bookButton.addActionListener(e -> bookAppointment());
addWindowListener(new WindowAdapter() {
    @Override public void windowClosing(WindowEvent e) {
        if (bookingInProgress) {
            int choice = JOptionPane.showConfirmDialog(AppointmentDesk.this,
                "A booking is in progress. Close anyway?", "Confirm close",
                JOptionPane.YES_NO_OPTION);
            if (choice == JOptionPane.YES_OPTION) dispose();
        } else dispose();
    }
});
```

Task 2.4 – Event trace and applet comparison



Order	Method / event	Purpose
1	main() → SwingUtilities.invokeLater(...)	Schedules GUI creation on the Event Dispatch Thread.
2	AppointmentDesk()	Constructs components, registers listeners, packs and positions frame.
3	setVisible(true) → windowOpened(WindowEvent)	Shows the window and initializes focus/data if required.
4	itemStateChanged(ItemEvent)	Department selection refreshes doctor choices.
5	actionPerformed(ActionEvent)	Book button validates and submits the appointment.
6	windowClosing(WindowEvent)	Checks bookingInProgress and asks for confirmation.
7	dispose() → windowClosed(WindowEvent)	Releases the native window resources.

Applet lifecycle is different: init() initializes the applet once; start() begins or resumes activity; paint(Graphics) draws its content; stop() pauses activity when it is no longer active; destroy() releases resources before removal. A JFrame is not governed by these applet lifecycle callbacks. It responds to construction, visibility and WindowEvents such as windowOpened, windowClosing and windowClosed.

Alternative concept – cached doctor map versus database fetch

Approach	Time per department change	Space
Load once into Map<String,List<Doctor>>	O(d) initial loading; O(m_d) to populate a department list (or O(1) lookup plus list rendering)	O(D + M) for D departments and M total doctor records
Fetch from database on every change	Network + SQL query latency each time; O(log M + m_d) with suitable index plus network latency	O(m_d) transient result data per change; lower persistent client cache

The map avoids repeated database round trips and is suitable when doctor data changes infrequently. If the data is frequently updated or very large, fetch on demand with caching and refresh rules. Database work should still run in SwingWorker rather than the Event Dispatch Thread.

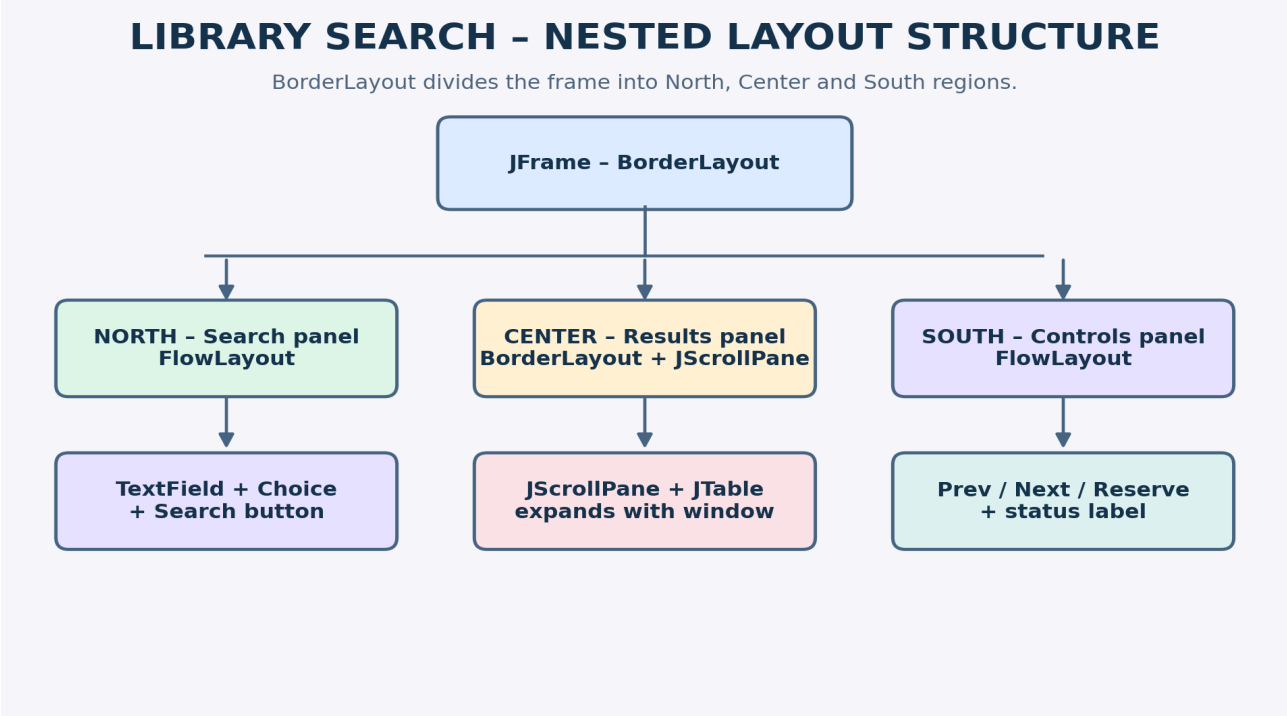
TASK 3 – FROM SKETCH TO LAYOUT: CAMPUS LIBRARY SEARCH SCREEN

Task 3.1 – Layout manager selection

Container / region	Layout manager	Reason
Main JFrame	BorderLayout	Separates North search row, Center results and South controls; Center receives remaining space.
Search row JPanel	FlowLayout(LEFT)	Keeps field, category selector and Search button in a compact horizontal row.
Results JPanel	BorderLayout	Places a JTable inside JScrollPane; scrollable table expands in the centre.
Bottom controls JPanel	FlowLayout(LEFT)	Prev, Next and Reserve buttons keep preferred sizes while remaining grouped.
Optional multi-screen container	CardLayout	Switches between search, book details and reservation confirmation cards.



Task 3.2 – Nested panel structure and designed screen



The mockup shows the title bar, a fixed-size search row, a central results area, bottom navigation/reservation buttons and a status message. It is a design illustration; the Java Swing code below builds the actual screen.

Task 3.3 – Complete implementation: LibrarySearchFrame.java

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.ArrayList;
import java.util.List;

public class LibrarySearchFrame extends JFrame {
    private final JTextField query = new JTextField(24);
    private final JComboBox<String> category =
            new JComboBox<>(new String[]{"All", "Title", "Author", "Subject"});
    private final DefaultTableModel model = new DefaultTableModel(
            new String[]{"Book ID", "Title", "Author", "Category", "Status"}, 0);
    private final JTable results = new JTable(model);
    private final JLabel status = new JLabel("Enter a search term.");
    private int page = 0;

    public LibrarySearchFrame() {
        super("Campus Library Search");
        setLayout(new BorderLayout(8, 8));

        JPanel searchPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        searchPanel.add(new JLabel("Search:"));
        searchPanel.add(query); searchPanel.add(category);
        JButton search = new JButton("Search");
        searchPanel.add(search);
        add(searchPanel, BorderLayout.NORTH);

        JPanel resultPanel = new JPanel(new BorderLayout());
        results.setFillsViewportHeight(true);
        resultPanel.add(new JScrollPane(results), BorderLayout.CENTER);
```

```
        add(resultPanel, BorderLayout.CENTER);

        JButton prev = new JButton("Prev");
        JButton next = new JButton("Next");
        JButton reserve = new JButton("Reserve");
        JPanel controls = new JPanel(new FlowLayout(FlowLayout.LEFT));
        controls.add(prev); controls.add(next); controls.add(reserve);
        controls.add(status);
        add(controls, BorderLayout.SOUTH);

        search.addActionListener(e -> performSearch());
        query.addActionListener(e -> performSearch());
        prev.addActionListener(e -> { if (page > 0) page--; performSearch(); });
        next.addActionListener(e -> { page++; performSearch(); });
        reserve.addActionListener(e -> reserveSelected());
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setMinimumSize(new Dimension(650, 400));
        setSize(850, 550);
        setLocationRelativeTo(null);
        loadDemoRows();
    }

    private void loadDemoRows() {
        model.addRow(new Object[]{"B101","Java Programming","Herbert
Schildt","Programming","Available"});
        model.addRow(new Object[]{"B102","Database Systems","Elmasri","Database","Issued"});
        model.addRow(new Object[]{"B103","Event-Driven
Design","Deitel","Programming","Available"});
        status.setText("3 sample records.");
    }

    private void performSearch() {
        String text = query.getText().trim();
        status.setText("Searching for: " + text + " | Page " + (page + 1));
        // Replace this demo filter with DAO search; run database work in SwingWorker.
    }

    private void reserveSelected() {
        int row = results.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Select a book first.");
            return;
        }
        JOptionPane.showMessageDialog(this, "Reservation request for " +
                model.getValueAt(row, 1) + " recorded.");
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new LibrarySearchFrame().setVisible(true));
    }
}
```

Task 3.4 – Resizing test plan and expected behaviour

Test size	Expected search row	Expected results area	Expected bottom row
Small: 650 × 400	Controls keep preferred sizes; may become crowded	Remains visible with scrollbars; reduced height/width	Buttons retain preferred sizes; status may wrap/clip if too small
Medium: 850 × 550	Same preferred component sizes	Expands to use additional centre space	Same button sizes and alignment
Large: 1200 × 800	Search row stays near preferred height	Grows in both dimensions and displays more rows	Stays at bottom; buttons do not stretch

To submit evidence, run the program locally and capture three screenshots at the stated sizes. The expected behaviour follows BorderLayout: North and South use preferred height, while Center receives the remaining space. The actual screenshot evidence must be captured from the running program.

Alternative concept – GridBagLayout or absolute positioning

Layout	Resizing behaviour	Time / space
Nested BorderLayout + FlowLayout	Predictable resizing; Center expands and fixed controls retain preferred sizes	Layout work is approximately O(c) for c components; uses a few nested panel objects O(p).
GridBagLayout	Very flexible constraints and weights; can achieve responsive resizing but requires careful constraints	Constraint calculation is more involved, commonly proportional to components O(c); similar component/panel memory O(c+p).
Absolute positioning (null + setBounds)	Does not adapt automatically; controls overlap or remain stranded when resized unless manually recalculated	Simple placement O(c) initially, but resize handling adds custom calculations; no layout metadata, though components still use O(c) memory.

The nested BorderLayout/FlowLayout solution is recommended for this screen because the required centre expansion is directly supported without custom resize calculations.

TASK 4 – EVENT-HANDLING FORENSICS: DRAWING PAD

Task 4.1 – Locate the defect

- The callback is misspelled as mouseclicked; the MouseListener method name is case-sensitive and must be mouseClicked(MouseEvent e).
- The incorrect method does not override the adapter method, so the listener receives clicks but the custom method is never called.
- The code never updates status.setText(...), so the label remains Clicks: 0.
- getGraphics() draws directly to the current screen surface; the drawing is not retained as application state.

Task 4.2 – Failing interaction

Click several points so dots appear, minimise the window, then restore it or cover and uncover it. The operating system requests repainting; the component is redrawn from its retained state. Since the original program did not store the points, the dots disappear. Direct drawing through getGraphics() is temporary and is not a replacement for paint().



Task 4.3 – Completed TODO 1 and TODO 2: DrawPad.java

```
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
import java.util.List;

public class DrawPad extends Frame {
    private final List<Point> points = new ArrayList<>();
    private final Label status = new Label("Clicks: 0");

    public DrawPad() {
        super("Draw Pad");
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            @Override public void mouseClicked(MouseEvent e) {
                points.add(e.getPoint());
                status.setText("Clicks: " + points.size());
                repaint();
            }
        });
```

```
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setSize(400, 300);
    }

    @Override public void paint(Graphics g) {
        super.paint(g);
        for (Point point : points)
            g.fillOval(point.x - 3, point.y - 3, 6, 6);
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> new DrawPad().setVisible(true));
    }
}
```

Task 4.4 – Prevent recurrence using @Override

The @Override annotation asks the compiler to verify that the method overrides a method from a superclass or implements an interface method. If mouseclicked is misspelled, compilation fails with an error similar to: method does not override or implement a method from a supertype. This exposes the defect before the application is run.

Required failing-compile experiment

```
@Override public void mouseclicked(MouseEvent e) {
    // Compile error: method does not override or implement a method
    // from a supertype.
}
```

For the assignment evidence, remove @Override and keep the misspelling once to demonstrate that the program compiles but the click callback is not invoked. Restore @Override and the correct mouseClicked spelling for the final program.

Alternative concept – off-screen BufferedImage

```
// Alternative drawing model: keep a persistent image buffer.
private BufferedImage canvas;
public void addDot(int x, int y) {
    Graphics2D g2 = canvas.createGraphics();
    g2.fillOval(x - 3, y - 3, 6, 6);
    g2.dispose();
    repaint();
}
@Override public void paint(Graphics g) {
    super.paint(g);
    if (canvas != null) g.drawImage(canvas, 0, 0, this);
}
```

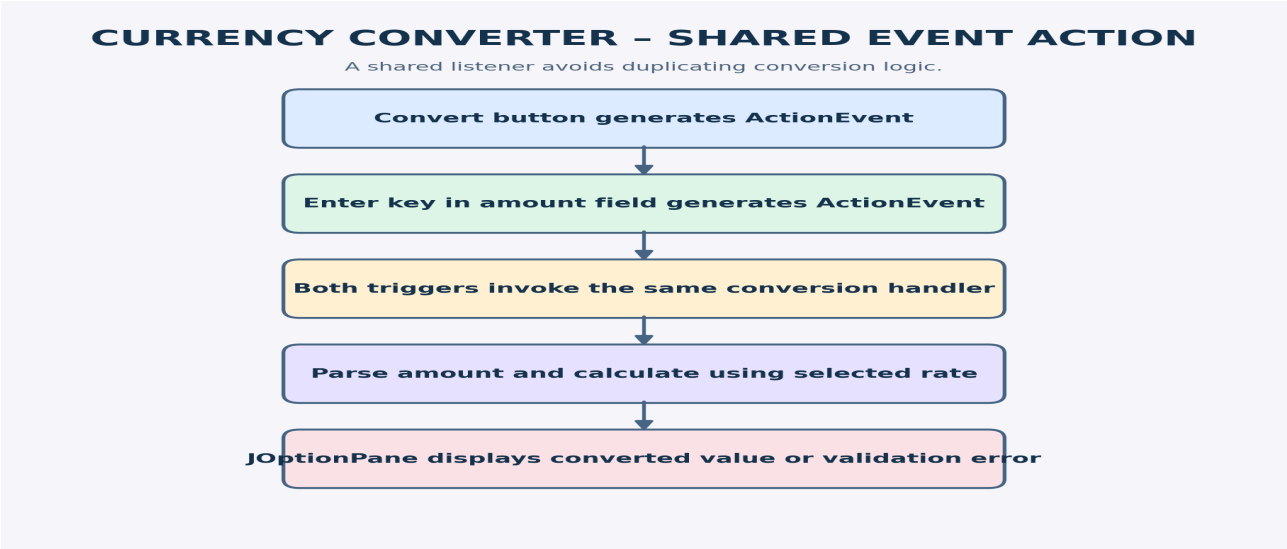
Approach	Time per repaint	Space
Store n points and redraw	O(n) draw operations per repaint	O(n) points; compact when only coordinates are needed
BufferedImage	O(width × height) to copy the image (typically optimized by graphics pipeline); new dot updates a small region	O(width × height × bytesPerPixel), e.g. 4 bytes per ARGB pixel

The point-list approach is suitable for simple dots and supports editing/undo by manipulating the model. BufferedImage is suitable for persistent raster painting and makes repainting a single image operation, but uses memory proportional to the canvas pixel count.

TASK 5 – SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES: CURRENCY CONVERTER

Task 5.1 – Four implementations

The converter accepts an INR amount, a target currency (USD or EUR), and converts using the fixed rates given in the question paper: USD = 0.012 and EUR = 0.011. Each version uses WindowAdapter for window closing. Both the Convert button and Enter key invoke the same conversion behaviour, and the result is displayed in a dialog.



Version A – Frame implements ActionListener

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.text.DecimalFormat;

public class ConverterFrameListener extends JFrame implements ActionListener {
    JTextField amount = new JTextField(10);
    JComboBox<String> target = new JComboBox<>(new String[]{"USD","EUR"});
    JButton button = new JButton("Convert");
    double rate(String c) { return c.equals("USD") ? .012 : .011; }

    ConverterFrameListener() {
        setTitle("Converter - Frame Listener"); setLayout(new FlowLayout());
        add(new JLabel("INR")); add(amount); add(target); add(button);
        button.addActionListener(this); amount.addActionListener(this);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE); pack();
    }
    public void actionPerformed(ActionEvent e) { convert(); }
    private void convert() {
        try {
            double value = Double.parseDouble(amount.getText().trim());
            if (!Double.isFinite(value) || value < 0) throw new NumberFormatException();
            String c = (String) target.getSelectedItem();
            JOptionPane.showMessageDialog(this,
                new DecimalFormat("0.00").format(value * rate(c)) + " " + c);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Invalid amount", "Error",
JOptionPane.ERROR_MESSAGE);
```

```
        }
    }
    public static void main(String[] a) { SwingUtilities.invokeLater(() -> new
ConverterFrameListener().setVisible(true)); }
}
```

Version B – Named inner class

```
class ConverterNamedInner extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD","EUR"});
    ConverterNamedInner() {
        setLayout(new FlowLayout()); JButton button = new JButton("Convert");
        add(amount); add(target); add(button);
        ConvertHandler handler = new ConvertHandler();
        button.addActionListener(handler); amount.addActionListener(handler);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE); pack();
    }
    private class ConvertHandler implements ActionListener {
        public void actionPerformed(ActionEvent e) { convert(); }
    }
    private void convert() {
        try {
            double n = Double.parseDouble(amount.getText().trim());
            if (!Double.isFinite(n) || n < 0) throw new NumberFormatException();
            String c=(String)target.getSelectedItem(); double r=c.equals("USD")?.012:.011;
            JOptionPane.showMessageDialog(this,String.format("%.2f %s",n*r,c));
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this,"Invalid amount","Error",JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Version C – Anonymous inner class

```
class ConverterAnonymous extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD","EUR"});
    ConverterAnonymous() {
        setLayout(new FlowLayout()); JButton button = new JButton("Convert");
        add(amount); add(target); add(button);
        ActionListener handler = new ActionListener() {
            @Override public void actionPerformed(ActionEvent e) { convert(); }
        };
        button.addActionListener(handler); amount.addActionListener(handler);
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE); pack();
    }
    private void convert() {
        try {
            double n=Double.parseDouble(amount.getText().trim());
            if (!Double.isFinite(n) || n < 0) throw new NumberFormatException();
            String c=(String)target.getSelectedItem();
            JOptionPane.showMessageDialog(this,String.format("%.2f
%s",n*(c.equals("USD")?.012:.011),c));
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this,"Invalid amount","Error",JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Version D – Lambda (completed TODO 1 and TODO 2)

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

public class Converter extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target = new JComboBox<>(new String[]{"USD", "EUR"});

    private static double rate(String currency) {
        return currency.equals("USD") ? 0.012 : 0.011;
    }

    public Converter() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        JButton convertButton = new JButton("Convert");
        add(new JLabel("INR")); add(amount); add(target);
        add(convertButton);
        ActionListener convertAction = e -> convert(); // one lambda listener
        convertButton.addActionListener(convertAction);
        amount.addActionListener(convertAction);       // Enter uses same listener
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) { dispose(); }
        });
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        pack(); setLocationRelativeTo(null);
    }

    private void convert() {
        try {
            double value = Double.parseDouble(amount.getText().trim());
            if (!Double.isFinite(value) || value < 0) throw new NumberFormatException();
            String currency = (String) target.getSelectedItem();
            String output = String.format("%.2f %s", value * rate(currency), currency);
            JOptionPane.showMessageDialog(this, output, "Converted amount",
                    JOptionPane.INFORMATION_MESSAGE);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Invalid amount", "Input error",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Converter().setVisible(true));
    }
}
```

Task 5.2 – Requirement change and code-change record

Style	Enter key	Dialog result	Approximate change
Frame implements ActionListener	Register this on amount field as well	Add JOptionPane in actionPerformed/convert	Small; same method handles both sources
Named inner class	Register same handler instance on amount	Handler calls shared convert() method	Small; reusable handler class unchanged
Anonymous inner class	Create one ActionListener variable and register twice	convert() shows JOptionPane	Small; anonymous object can be reused if stored in a variable
Lambda	Store lambda in ActionListener variable; register on button and field	convert() displays JOptionPane	Very small; one expression shared by both controls

Task 5.3 – Comparison

Style	Readability	Access / reuse	Best fit

Frame implements ActionListener	Simple for tiny forms; frame accumulates event code	Direct access to fields; handler tied to frame	Small single-purpose window
Named inner class	Clear name and separation; more lines	Direct access to outer fields; can be reused within class	Large handler logic or multiple related actions
Anonymous inner class	Self-contained but verbose	Direct outer-field access; reuse only if stored	One-off multi-line listener
Lambda	Concise for one-method listener	Direct lexical access; easy to share one ActionListener variable	Short actions and modern Swing code

For a large application, use named handler/service classes or Swing Action objects rather than putting all event logic in the frame. They provide testability, reuse and clearer separation. A method reference (this::convert) can replace a simple lambda when the signature matches.

Alternative concept – method reference or shared Action

```
Action convertAction = new AbstractAction("Convert") {
    @Override public void actionPerformed(ActionEvent e) { convert(); }
};
convertButton.setAction(convertAction);
amount.addActionListener(convertAction);
getRootPane().getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW)
    .put(KeyStroke.getKeyStroke("ctrl ENTER"), "convert");
getRootPane().getActionMap().put("convert", convertAction);
```

One shared Action can be attached to a button, menu item and keyboard shortcut, keeping enable/disable state and command behaviour synchronized. A method reference this::convert is concise but may require a wrapper if convert() has no ActionEvent parameter.

Approach	Listener/action objects	Adding a new trigger
Separate listener per trigger	Potentially O(t) objects for t triggers	Register another listener and possibly duplicate logic
One shared lambda/listener	O(1) shared handler object for the action	Register same object on new source
One Swing Action	O(1) Action object per command	Attach Action to button/menu; map keyboard shortcut

TASK 6 – DESIGN CHALLENGE: CAMPUS EVENT REGISTRATION DESKTOP APPLICATION

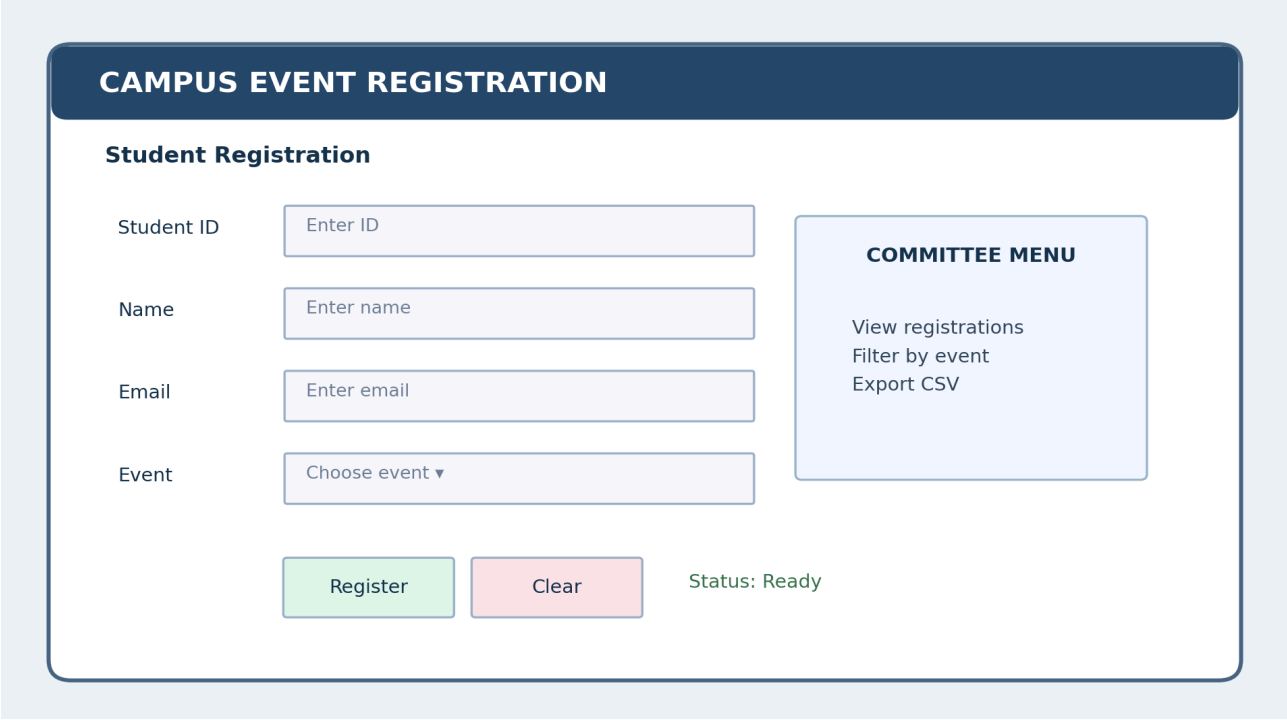
Stage 1 – Interpret the problem

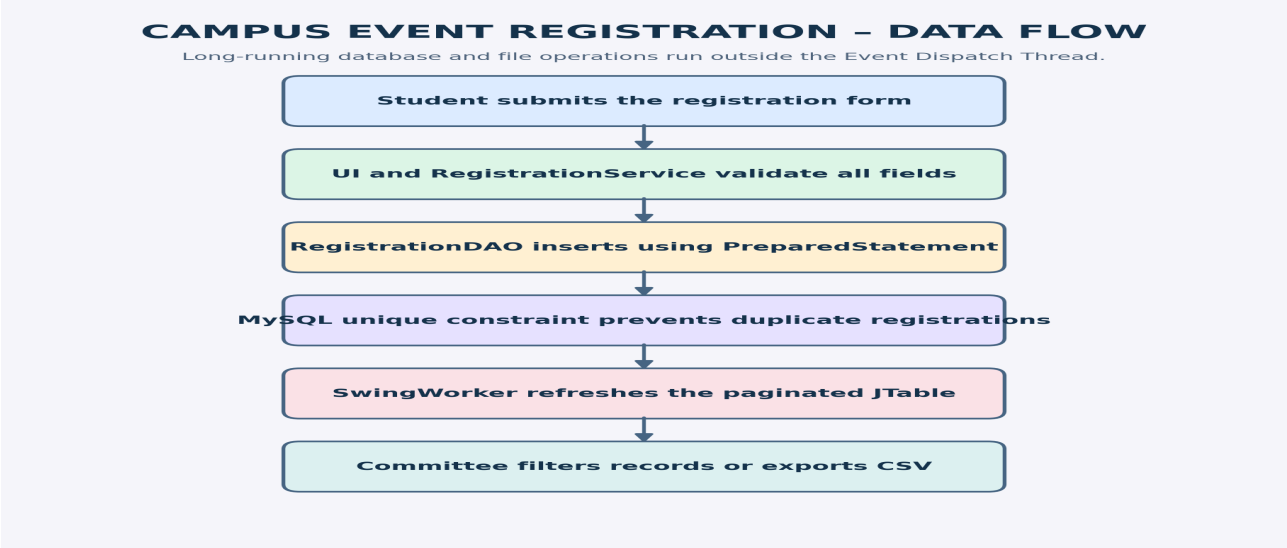
User	Screens / actions	Data and rules
Student	Registration form; register; clear; confirmation	Student ID, name, email, selected event; one registration per student per event
Committee member	Registration table; event filter; export list	Can view, filter and export registrations; access controlled in a real deployment
Administrator	Event setup / maintenance (optional extension)	Event ID, name, date, capacity and active status

Testable requirements

- The form rejects blank student ID, blank name, malformed email and missing event selection.
- A student can register for an event only once; duplicate attempts show a clear message.
- The database has a UNIQUE(student_id, event_id) constraint as the final duplicate-prevention authority.
- Registration, filtering and export database operations do not execute on the Swing Event Dispatch Thread.
- The table loads one page at a time using SQL LIMIT and OFFSET.
- The committee can filter registrations by event and export the displayed matching records to CSV.
- After successful registration, the form displays confirmation and the table refreshes.

Stage 2 – Screen and event flow



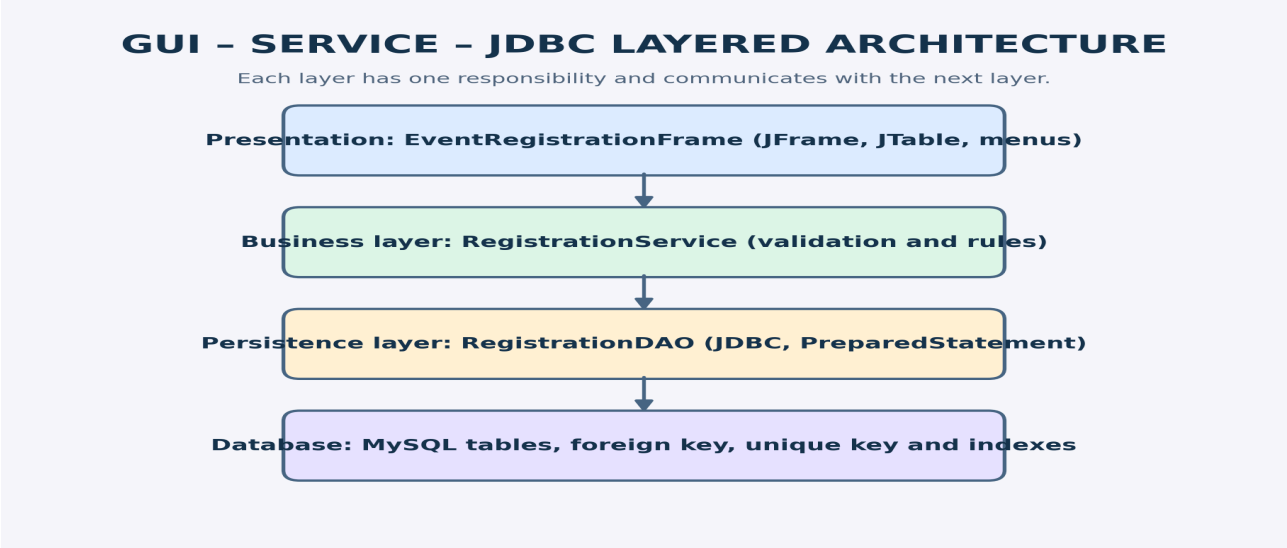


The menu bar contains File → Export CSV and Exit, and View → Refresh. Registration success/error is shown using JOptionPane dialogs. The committee table can be displayed in a separate tab or a second card in the same JFrame.

Stage 3 – Critical design decisions

Concern	Where handled	Design decision
Input validation	UI for immediate feedback; RegistrationService for authoritative rules	Trim fields, validate required values and email pattern before DAO call.
Duplicate checking	Database constraint plus service-level friendly message	UNIQUE(student_id,event_id); catch SQLState 23000 / duplicate-key error.
Database access	RegistrationDAO	PreparedStatement and try-with-resources; no SQL in UI class.
Responsiveness	SwingWorker	doInBackground() runs JDBC; done() updates table/status on EDT.
Large table	DAO pagination	LIMIT ? OFFSET ?; page size fixed, next/previous navigation.
Export	DAO query + file writer in SwingWorker	Escape CSV fields and do not freeze UI.

Stage 4 – Class diagram and handlers



Class	Layer	Main methods
EventRegistrationFrame	Presentation	buildUI(), registerAction(), filterAction(), loadPage(), exportAction()
RegistrationService	Business	validate(), register(), listEvents(), getRegistrations()
RegistrationDAO	Persistence	insert(), findDuplicate(), fetchPage(), countByEvent(), fetchForExport()
Registration	Domain model	studentId, studentName, email, eventId, eventName, registeredAt

Pseudocode – Register action

```
START
    READ studentID, studentName, email, selectedEvent
    IF any required field is empty THEN
        DISPLAY validation message
        STOP
    END IF
    IF email format is invalid THEN
        DISPLAY "Enter a valid email"
        STOP
    END IF
    CALL RegistrationService.register(...)
    SERVICE validates values
    DAO INSERTS using PreparedStatement
    IF UNIQUE constraint violation THEN
        DISPLAY "Already registered for this event"
    ELSE
        DISPLAY confirmation number
        REFRESH current table page
    END IF
END
```

Pseudocode – Filter action

```
START
    READ selectedEventFilter and pageNumber
    SET loading indicator visible
    CREATE SwingWorker
        DO IN BACKGROUND
            CALL DAO.fetchPage(eventFilter, pageSize, offset)
            CALL DAO.countByEvent(eventFilter)
```

```
        DONE ON EVENT DISPATCH THREAD
            REPLACE JTable model rows
            UPDATE page label and navigation buttons
            HIDE loading indicator
    END WORKER
END
```

Stage 5 – SQL script: schema.sql

```
CREATE DATABASE IF NOT EXISTS campus_events;
USE campus_events;

CREATE TABLE IF NOT EXISTS events (
    event_id INT PRIMARY KEY AUTO_INCREMENT,
    event_name VARCHAR(120) NOT NULL UNIQUE,
    event_date DATE NOT NULL,
    capacity INT NOT NULL DEFAULT 100,
    active BOOLEAN NOT NULL DEFAULT TRUE,
    CHECK (capacity > 0)
);

CREATE TABLE IF NOT EXISTS registrations (
    registration_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    student_id VARCHAR(30) NOT NULL,
    student_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL,
    event_id INT NOT NULL,
    registered_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_registration_event
        FOREIGN KEY (event_id) REFERENCES events(event_id),
    CONSTRAINT uq_student_event UNIQUE (student_id, event_id)
);

CREATE INDEX idx_registrations_event
    ON registrations(event_id, registered_at);

INSERT INTO events(event_name, event_date, capacity) VALUES
('Cultural Fest', '2026-12-10', 500),
('Tech Symposium', '2026-12-15', 300),
('Music Night', '2026-12-20', 250);
```

RegistrationDAO.java – JDBC operations and pagination

```
import java.sql.*;
import java.util.*;

public class RegistrationDAO {
    private static final String URL = "jdbc:mysql://localhost:3306/campus_events";
    private static final String USER = "root";
    private static final String PASSWORD = "CHANGE_ME";

    public void insert(Registration r) throws SQLException {
        String sql = "INSERT INTO registrations(student_id,student_name,email,event_id)
VALUES(?,?,?,?)";
        try (Connection c = DriverManager.getConnection(URL, USER, PASSWORD);
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, r.studentId);
            ps.setString(2, r.studentName);
            ps.setString(3, r.email);
            ps.setInt(4, r.eventId);
            ps.executeUpdate();
        }
    }

    public List<Registration> fetchPage(Integer eventId, int limit, int offset)
            throws SQLException {
        String base = "SELECT r.registration_id,r.student_id,r.student_name,r.email," +
                "r.event_id,e.event_name,r.registered_at FROM registrations r " +
```

```
                "JOIN events e ON e.event_id=r.event_id ";
        String where = eventId == null ? "" : "WHERE r.event_id=? ";
        String sql = base + where + "ORDER BY r.registered_at DESC LIMIT ? OFFSET ?";
        List<Registration> list = new ArrayList<>();
        try (Connection c = DriverManager.getConnection(URL, USER, PASSWORD);
             PreparedStatement ps = c.prepareStatement(sql)) {
            int i = 1;
            if (eventId != null) ps.setInt(i++, eventId);
            ps.setInt(i++, limit); ps.setInt(i, offset);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(new Registration(rs.getLong("registration_id"),
                        rs.getString("student_id"), rs.getString("student_name"),
                        rs.getString("email"), rs.getInt("event_id"),
                        rs.getString("event_name"), rs.getTimestamp("registered_at")));
                }
            }
        }
        return list;
    }

    public int count(Integer eventId) throws SQLException {
        String sql = "SELECT COUNT(*) FROM registrations" +
                (eventId == null ? "" : " WHERE event_id=?");
        try (Connection c = DriverManager.getConnection(URL, USER, PASSWORD);
             PreparedStatement ps = c.prepareStatement(sql)) {
            if (eventId != null) ps.setInt(1, eventId);
            try (ResultSet rs = ps.executeQuery()) {
                rs.next(); return rs.getInt(1);
            }
        }
    }
}
```

Registration.java and RegistrationService.java

```
import java.sql.Timestamp;
class Registration {
    final long id; final String studentId, studentName, email, eventName;
    final int eventId; final Timestamp registeredAt;

    Registration(String studentId, String studentName, String email, int eventId) {
        this(0, studentId, studentName, email, eventId, "", null);
    }
    Registration(long id, String studentId, String studentName, String email,
                 int eventId, String eventName, Timestamp registeredAt) {
        this.id=id; this.studentId=studentId; this.studentName=studentName;
        this.email=email; this.eventId=eventId; this.eventName=eventName;
        this.registeredAt=registeredAt;
    }
}

class RegistrationService {
    private final RegistrationDAO dao = new RegistrationDAO();
    void register(String id, String name, String email, int eventId) throws Exception {
        if (id == null || id.trim().isEmpty()) throw new IllegalArgumentException("Student ID is
required.");
        if (name == null || name.trim().isEmpty()) throw new IllegalArgumentException("Student name
is required.");
        if (email == null || !email.trim().matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$"))
            throw new IllegalArgumentException("Enter a valid email.");
        if (eventId <= 0) throw new IllegalArgumentException("Select an event.");
        dao.insert(new Registration(id.trim(), name.trim(), email.trim(), eventId));
    }
}
```

EventRegistrationFrame.java – Swing GUI, SwingWorker and CSV export

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.sql.SQLException;
import java.util.List;

public class EventRegistrationFrame extends JFrame {
    private final JTextField studentId = new JTextField(10);
    private final JTextField studentName = new JTextField(14);
    private final JTextField email = new JTextField(16);
    private final JComboBox<EventItem> eventBox = new JComboBox<>(new EventItem[]{
        new EventItem(1, "Cultural Fest"), new EventItem(2, "Tech Symposium"),
        new EventItem(3, "Music Night")
    });
    private final JComboBox<EventItem> filterBox = new JComboBox<>();
    private final DefaultTableModel model = new DefaultTableModel(
        new String[]{"ID","Student ID","Name","Email","Event","Registered At"},0);
    private final JTable table = new JTable(model);
    private final JLabel status = new JLabel("Ready");
    private final RegistrationService service = new RegistrationService();
    private final RegistrationDAO dao = new RegistrationDAO();
    private int page = 0;
    private static final int PAGE_SIZE = 25;

    public EventRegistrationFrame() {
        super("Campus Event Registration");
        setLayout(new BorderLayout(8,8));
        JMenuBar bar = new JMenuBar();
        JMenu file = new JMenu("File");
        JMenuItem export = new JMenuItem("Export CSV");
        JMenuItem exit = new JMenuItem("Exit");
        file.add(export); file.add(exit); bar.add(file);
        JMenu view = new JMenu("View");
        JMenuItem refresh = new JMenuItem("Refresh");
        view.add(refresh); bar.add(view); setJMenuBar(bar);
        export.addActionListener(e -> exportCsv());
        exit.addActionListener(e -> dispose());
        refresh.addActionListener(e -> loadPage());

        JPanel form = new JPanel(new FlowLayout(FlowLayout.LEFT));
        form.setBorder(BorderFactory.createTitledBorder("Student Registration"));
        form.add(new JLabel("Student ID")); form.add(studentId);
        form.add(new JLabel("Name")); form.add(studentName);
        form.add(new JLabel("Email")); form.add(email);
        form.add(new JLabel("Event")); form.add(eventBox);
        JButton register = new JButton("Register");
        form.add(register);
        add(form, BorderLayout.NORTH);

        JPanel listing = new JPanel(new BorderLayout(5,5));
        JPanel filter = new JPanel(new FlowLayout(FlowLayout.LEFT));
        filter.add(new JLabel("Filter event:"));
        filterBox.addItem(new EventItem(0, "All events"));
        filterBox.addItem(new EventItem(1, "Cultural Fest"));
        filterBox.addItem(new EventItem(2, "Tech Symposium"));
        filterBox.addItem(new EventItem(3, "Music Night"));
        JButton apply = new JButton("Apply");
        JButton prev = new JButton("Previous");
        JButton next = new JButton("Next");
        filter.add(filterBox); filter.add(apply); filter.add(prev); filter.add(next);
        listing.add(filter, BorderLayout.NORTH);
        listing.add(new JScrollPane(table), BorderLayout.CENTER);
```

```
        add(listing, BorderLayout.CENTER);
        add(status, BorderLayout.SOUTH);

        register.addActionListener(e -> registerStudent());
        apply.addActionListener(e -> { page=0; loadPage(); });
        prev.addActionListener(e -> { if(page>0) page--; loadPage(); });
        next.addActionListener(e -> { page++; loadPage(); });

        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(1050,600); setLocationRelativeTo(null);
        loadPage();
    }

    private Integer selectedFilter() {
        EventItem item=(EventItem)filterBox.getSelectedItem();
        return item==null || item.id==0 ? null : item.id;
    }

    private void registerStudent() {
        EventItem event=(EventItem)eventBox.getSelectedItem();
        if(event==null) { showError("Select an event."); return; }
        setBusy(true);
        new SwingWorker<Void,Void>() {
            protected Void doInBackground() throws Exception {
                service.register(studentId.getText(), studentName.getText(),
                        email.getText(), event.id);
                return null;
            }
            protected void done() {
                setBusy(false);
                try {
                    get();
                    JOptionPane.showMessageDialog(EventRegistrationFrame.this,
                            "Registration successful.");
                    loadPage();
                } catch (Exception ex) {
                    Throwable cause=ex.getCause();
                    String msg=cause==null?ex.getMessage():cause.getMessage();
                    if (cause instanceof SQLException &&
                        "23000".equals(((SQLException)cause).getSQLState()))
                        msg="Already registered for this event.";
                    showError(msg);
                }
            }
        }.execute();
    }

    private void loadPage() {
        setBusy(true);
        Integer eventId=selectedFilter();
        int offset=page*PAGE_SIZE;
        new SwingWorker<List<Registration>,Void>() {
            protected List<Registration> doInBackground() throws Exception {
                return dao.fetchPage(eventId,PAGE_SIZE,offset);
            }
            protected void done() {
                setBusy(false);
                try {
                    List<Registration> rows=get();
                    model.setRowCount(0);
                    for(Registration r:rows)
                        model.addRow(new Object[]{r.id,r.studentId,r.studentName,
                            r.email,r.eventName,r.registeredAt});
                    status.setText("Page "+(page+1)+" | Showing "+rows.size()+" rows");
                } catch(Exception ex) { showError("Could not load registrations:
"+ex.getMessage()); }
```

```
            }
        }.execute();
    }

    private void exportCsv() {
        JFileChooser chooser=new JFileChooser();
        if(chooser.showSaveDialog(this)!=JFileChooser.APPROVE_OPTION) return;
        Path target=chooser.getSelectedFile().toPath();
        Integer eventId=selectedFilter();
        new SwingWorker<Void,Void>() {
            protected Void doInBackground() throws Exception {
                List<Registration> rows=dao.fetchPage(eventId,100000,0);
                try(BufferedWriter w=Files.newBufferedWriter(target,StandardCharsets.UTF_8)) {
                    w.write("Registration ID,Student ID,Name,Email,Event,Registered At\n");
                    for(Registration r:rows) {
                        w.write(r.id+","+csv(r.studentId)+","+csv(r.studentName)+","+
csv(r.email)+","+csv(r.eventName)+","+csv(String.valueOf(r.registeredAt))+"\n");
                    }
                }
                return null;
            }
            protected void done() {
                try { get(); JOptionPane.showMessageDialog(EventRegistrationFrame.this,"CSV
exported."); }
                catch(Exception ex) { showError("Export failed: "+ex.getMessage()); }
            }
        }.execute();
    }

    private String csv(String value) {
        String s=value==null?"":value.replace("\"","\"\"");
        return "\""+s+"\"";
    }
    private void setBusy(boolean busy) {
        status.setText(busy?"Working...":"Ready");
    }
    private void showError(String message) {
        JOptionPane.showMessageDialog(this,message,"Error",JOptionPane.ERROR_MESSAGE);
    }

    static class EventItem {
        final int id; final String name;
        EventItem(int id,String name){this.id=id;this.name=name;}
        public String toString(){return name;}
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new EventRegistrationFrame().setVisible(true));
    }
}
```

Database configuration and execution

Add MySQL Connector/J to the classpath and update RegistrationDAO credentials. Compile: javac *.java. Run: java EventRegistrationFrame. Execute schema.sql in MySQL first. The example seeds three event IDs and uses a database uniqueness constraint to reject duplicates even if two users submit simultaneously. In production, load event choices from the events table rather than hard-coding the sample list.

```
# Example terminal commands (adjust connector JAR path)
javac -cp .:mysql-connector-j-<version>.jar *.java
java  -cp .:mysql-connector-j-<version>.jar EventRegistrationFrame
```

Demonstration test cases

Test	Input / action	Expected result
Normal	STU001, Asha, asha@example.com, Cultural Fest	One row inserted; success dialog; table refreshed.
Invalid	Blank student ID or malformed email	Validation message; no database insert.
Duplicate	Submit STU001 for Cultural Fest again	Unique constraint rejects insert; friendly duplicate message.
Filter	Choose Tech Symposium and Apply	Only registrations for selected event appear, page by page.
Export	File → Export CSV	CSV file contains filtered registrations.

Alternative concept – load all 100,000 rows versus paginated SwingWorker

Approach	Loading time	Memory / responsiveness
Load all rows at once	O(N) rows plus database/network transfer; large initial query and model construction	O(N × average row size) client memory; EDT may freeze if done on EDT; long wait before first display.
LIMIT/OFFSET page size P in SwingWorker	O(P) transfer per page plus query/index cost; first page arrives sooner	O(P × average row size) table memory; background query keeps EDT responsive.

For N = 100,000 registrations and P = 25, the client table holds about 25 rows rather than 100,000 at a time, reducing row-object memory by roughly a factor of 4,000 in the simple row-count comparison. Actual bytes depend on string lengths and JVM object overhead. OFFSET pagination can become slower for very large offsets; keyset pagination using the last registration ID/timestamp is a scalable alternative. SwingWorker keeps JDBC and file I/O off the event thread.

Design defence

The design separates Swing components from validation and JDBC persistence. The service validates input; the database unique key guarantees duplicate prevention; DAO methods use PreparedStatement and close resources; SwingWorker keeps slow operations away from the Event Dispatch Thread; JTable pagination limits client memory; and CSV export is performed in a background worker. The UI provides clear success, invalid-input and duplicate messages.

FINAL CONCLUSION

Assignment 4 demonstrates event-driven design from requirements through implementation. Task 1 repairs a fee-payment interface by separating UI, service and JDBC layers. Task 2 models appointment events and window closing. Task 3 uses nested layout managers so the results region expands correctly. Task 4 fixes the mouse callback and stores drawing state for repainting. Task 5 compares four listener styles and reuses one action for button and Enter-key triggers. Task 6 designs a layered Swing–JDBC registration application with validation, duplicate protection, pagination, asynchronous loading and CSV export.

GITHUB REPOSITORY STRUCTURE

```
CSA09-<RegisterNumber>-Assignment4/
├── README.md
├── Task01_FeePayment/
│   ├── FeeWindow.java
│   ├── FeeService.java
│   ├── FeeDAO.java
│   └── screenshots/
├── Task02_AppointmentDesk/
│   ├── AppointmentDesk.java
│   └── screenshots/
├── Task03_LibrarySearch/
│   ├── LibrarySearchFrame.java
│   └── screenshots/
├── Task04_DrawPad/
│   ├── DrawPad.java
│   └── screenshots/
├── Task05_CurrencyConverter/
│   ├── ConverterFrameListener.java
│   ├── ConverterNamedInner.java
│   ├── ConverterAnonymous.java
│   ├── Converter.java
│   └── screenshots/
└── Task06_EventRegistration/
    ├── EventRegistrationFrame.java
    ├── Registration.java
    ├── RegistrationService.java
    ├── RegistrationDAO.java
    ├── schema.sql
    └── screenshots/
```